

MAINTENANCE

- Maintenance of any software:
 - Requires much more effort than the effort to develop the product itself.
 - Development effort to maintenance effort is typically 40:60.

TYPES OF MAINTENANCE?

- **Corrective maintenance:**
 - Correct errors which were not discovered during the product development phases.
- **Perfective maintenance:**
 - Improve implementation of the system
 - enhance functionalities of the system.
- **Adaptive maintenance:**
 - Port software to a new environment,
 - e.g. to a new computer or to a new operating system.

ITERATIVE WATERFALL MODEL

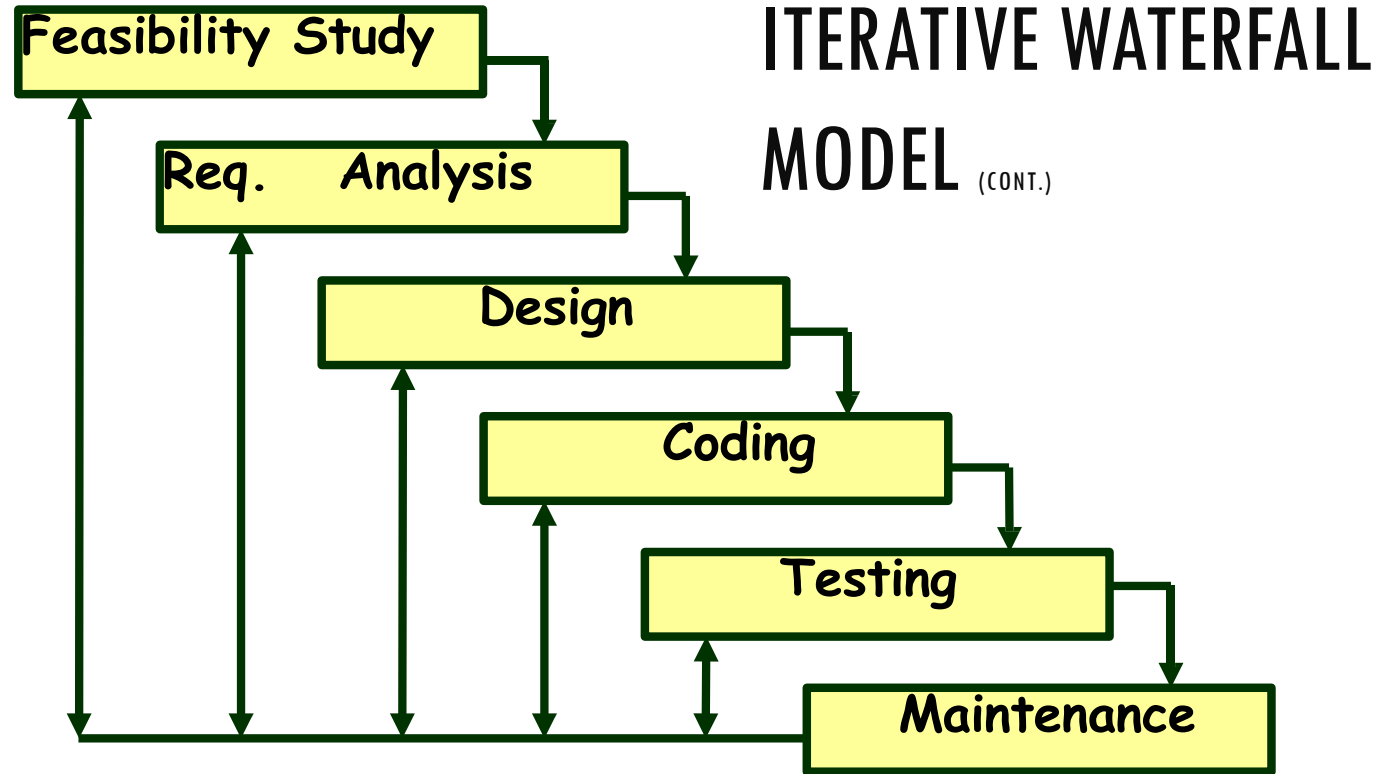
- Classical waterfall model is idealistic:
 - Assumes that no defect is introduced during any development activity.
 - In practice:
 - Defects do get introduced in almost every phase of the life cycle.

ITERATIVE WATERFALL MODEL (CONT.)

- Defects usually get detected much later in the life cycle:
 - For example, a design defect might go unnoticed till the coding or testing phase.
 - The later the phase in which the defect gets detected, the more expensive is its removal --- why?**

ITERATIVE WATERFALL MODEL (CONT.)

- Once a defect is detected:
 - The phase in which it occurred needs to be reworked.
 - Redo some of the work done during that and all subsequent phases.
- Therefore need feedback paths in the classical waterfall model.

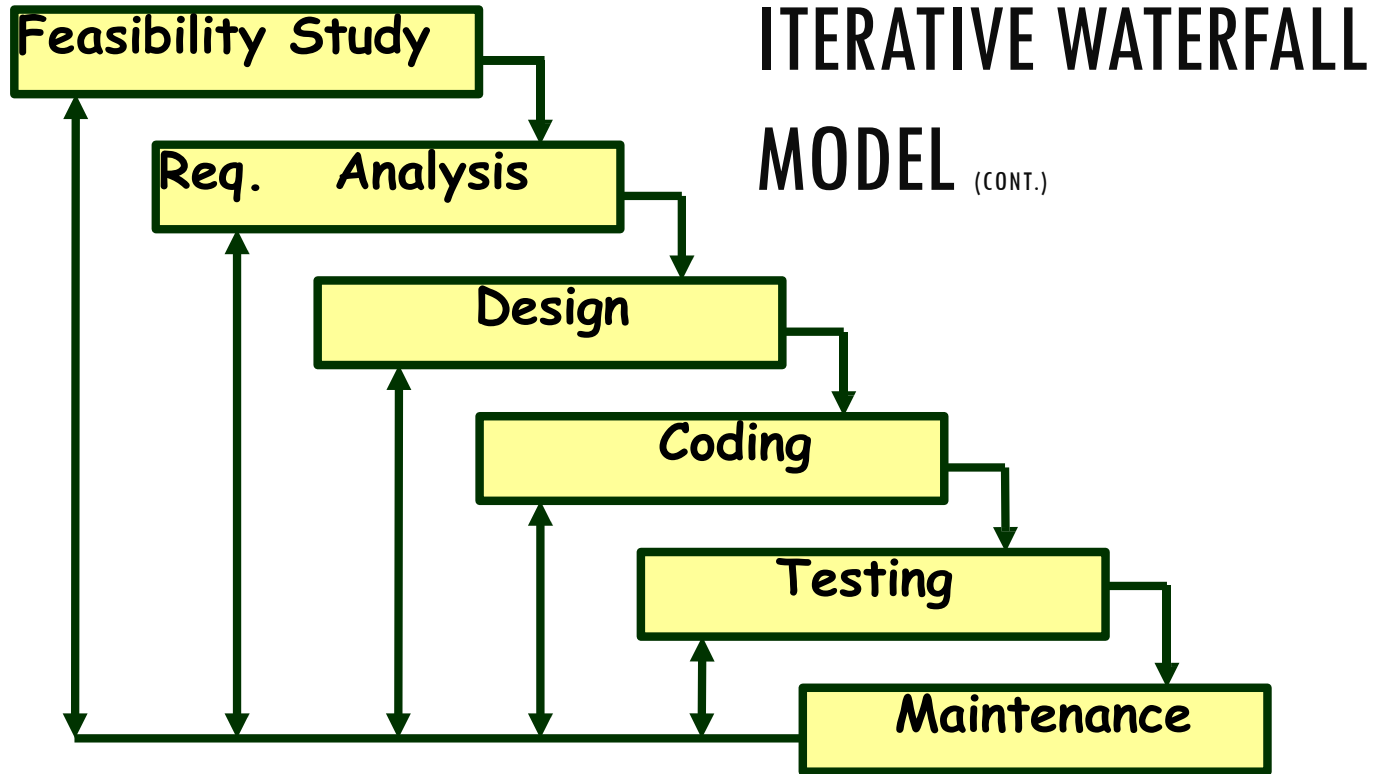


PHASE CONTAINMENT OF ERRORS (CONT...)

- **Errors should be detected:**
 - **In the same phase in which they are introduced.**
- For example:
 - If a design problem is detected in the design phase itself,
 - The problem can be taken care of much more easily
 - Than say if it is identified at the end of the integration and system testing phase.

PHASE CONTAINMENT OF ERRORS

- Reason: rework must be carried out not only to the design but also to code and test phases.
- The principle of detecting errors as close to its point of introduction as possible:
 - is known as **phase containment of errors.**
- Iterative waterfall model is by far the most widely used model.
 - Almost every other model is derived from the waterfall model.



WATERFALL STRENGTHS

- Easy to understand, easy to use, especially by inexperienced staff
- Milestones are well understood by the team
- Provides requirements stability during development
- Facilitates strong management control (plan, staff, track)

WATERFALL DEFICIENCIES

- All requirements must be known upfront – **in most projects requirement change occurs after project start**
- Can give a false impression of progress
- Integration is one big bang at the end
- Little opportunity for customer to pre-view the system.

WHEN TO USE THE WATERFALL MODEL?

- Requirements are well known and stable
- Technology is understood
- Development team have experience with similar projects

CLASSICAL WATERFALL MODEL (CONT.)

- Irrespective of the life cycle model actually followed:
 - The documents should reflect a classical waterfall model of development.
 - Facilitates comprehension of the documents.**

CLASSICAL WATERFALL MODEL (CONT.)

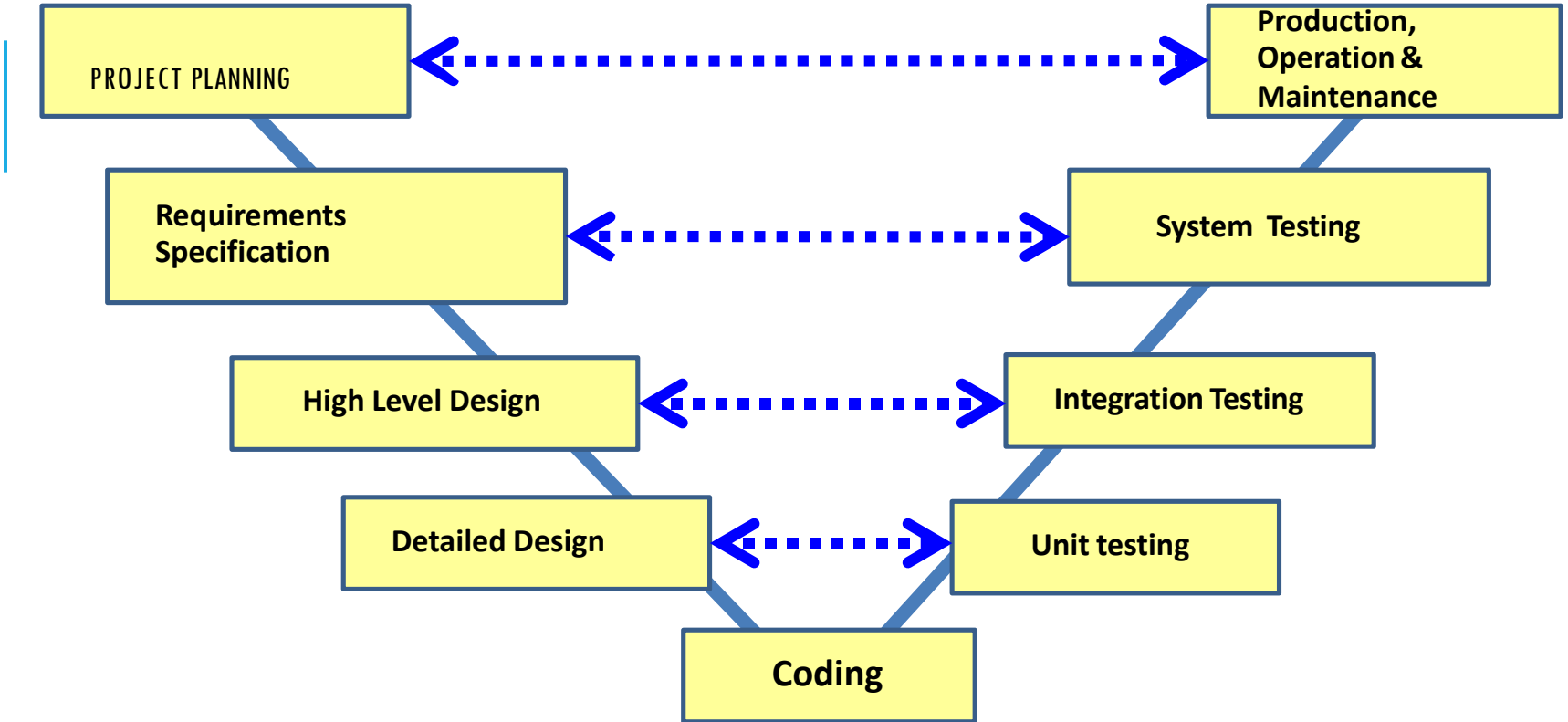
- Metaphor of mathematical theorem proving:
 - A mathematician presents a proof as a single chain of deductions,
 - Even though the proof might have come from a convoluted set of partial attempts, blind alleys and backtracks.



V MODEL

V MODEL

- It is a variant of the Waterfall
 - emphasizes verification and validation
 - V&V activities are spread over the entire life cycle.
- In every phase of development:
 - Testing activities are planned in parallel with development.



V MODEL

STEPS

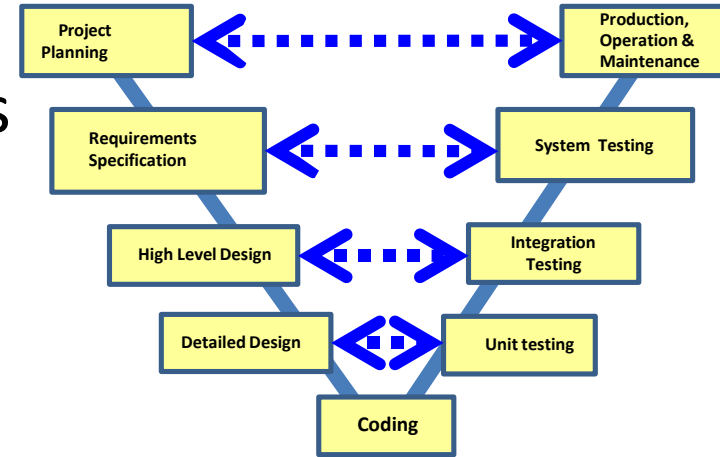
- **Planning**
- **Requirements Analysis and Specification**
 - **System test design**
- **High-level Design**
 - **Integration Test design**
- **Detailed Design**
 - **Unit test design**

V MODEL: STRENGTHS

- Starting from early stages of software development:
 - **Emphasizes planning for verification and validation of the software**
- Each deliverable is made testable
- Easy to use

V MODEL WEAKNESSES

- Does not support overlapping of phases
- Does not handle iterations or phases
- Does not easily accommodate later changes to requirements
- Does not provide support for effective risk handling



WHEN TO USE V MODEL

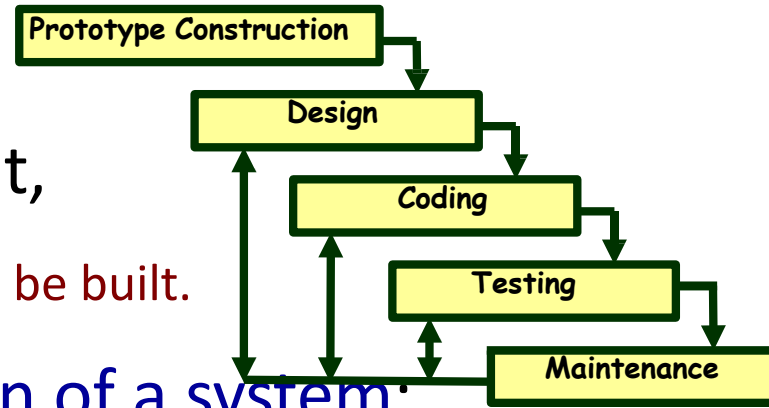
- Natural choice for systems requiring high reliability:
 - Embedded control applications, safety-critical software
- All requirements are known up-front
- Solution and technology are known



PROTOTYPING MODEL

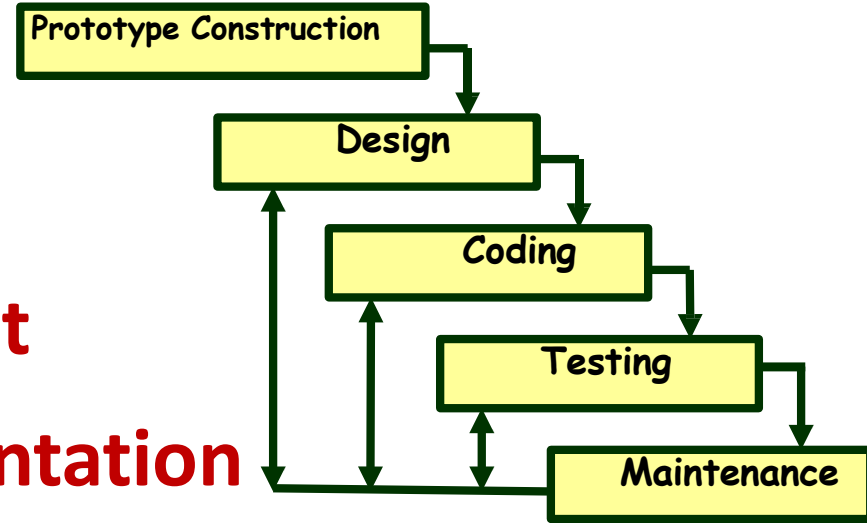
PROTOTYPING MODEL

- A derivative of waterfall model.
- Before starting actual development,
 - A working prototype of the system should first be built.
- A prototype is a toy implementation of a system:
 - Limited functional capabilities,
 - Low reliability,
 - Inefficient performance.



REASONS FOR PROTOTYPING

- **Learning by doing:** useful where requirements are only partially known
- **Improved communication**
- **Improved user involvement**
- **Reduced need for documentation**
- **Reduced maintenance costs**



REASONS FOR DEVELOPING A PROTOTYPE

- **Illustrate to the customer:**

- input data formats, messages, reports, or interactive dialogs.

- **Examine technical issues associated with product development:**

- Often major design decisions depend on issues like:

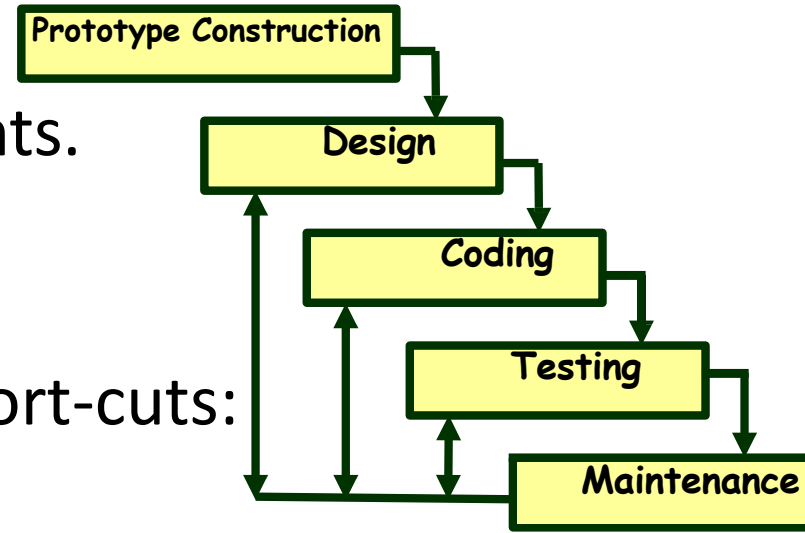
- Response time of a hardware controller,
 - Efficiency of a sorting algorithm, etc.

PROTOTYPING MODEL (CONT.)

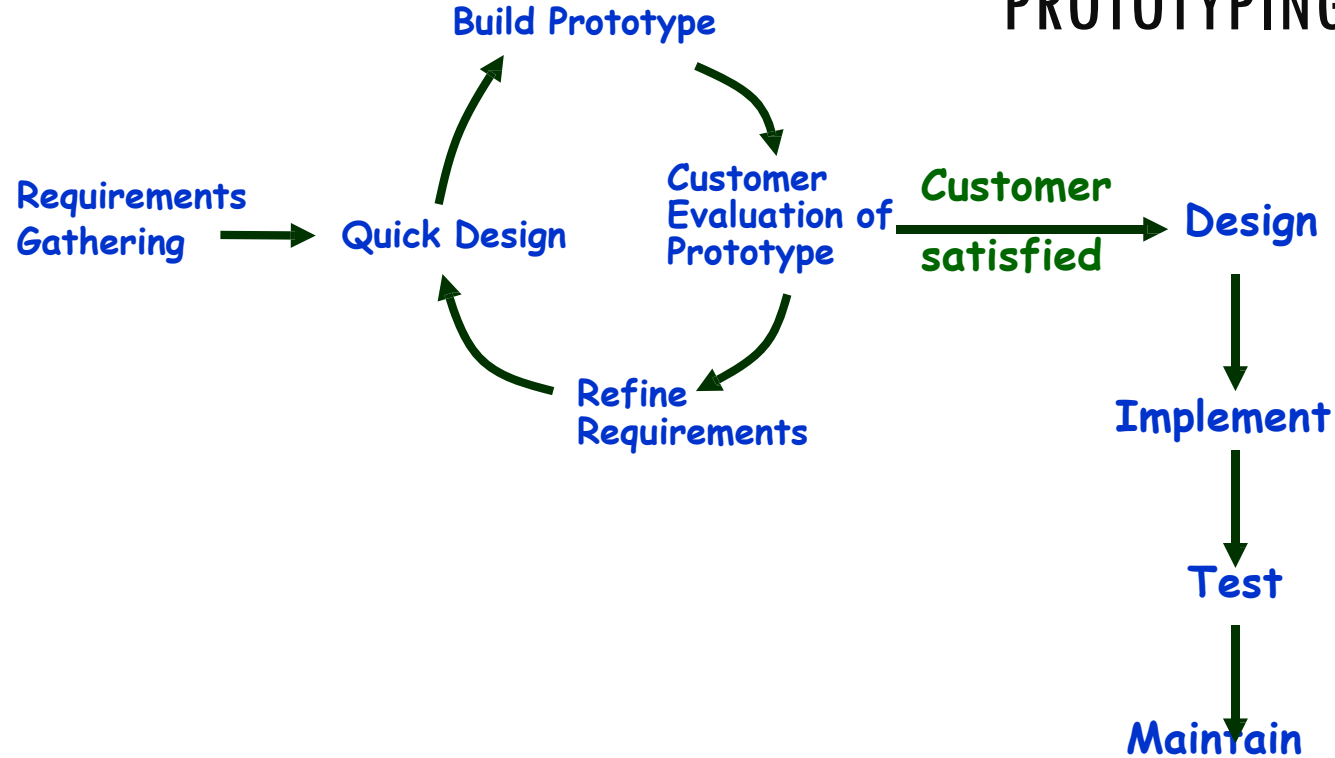
- Another reason for developing a prototype:
 - **It is impossible to “get it right” the first time,**
 - We must plan to throw away the first version:
 - If we want to develop a good software.

PROTOTYPING MODEL

- Start with approximate requirements.
- Carry out a quick design.
- Prototype is built using several short-cuts:
 - Short-cuts might involve:
 - Using inefficient, inaccurate, or dummy functions.
 - A table look-up rather than performing the actual computations.

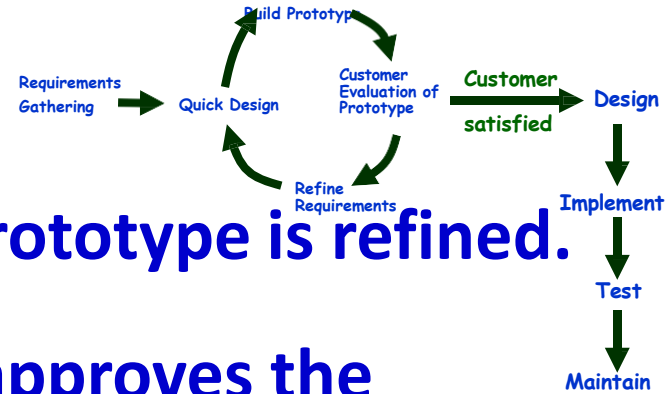


PROTOTYPING MODEL (CONT.)



PROTOTYPING MODEL (CONT.)

- The developed prototype is submitted to the customer for his evaluation:



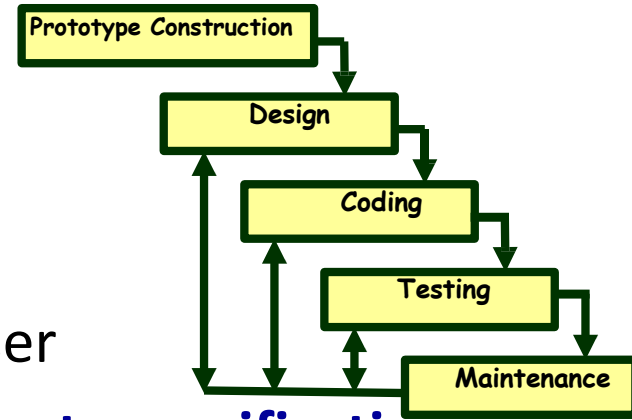
- Based on the user feedback, the prototype is refined.
- This cycle continues until the user approves the prototype.

- The actual system is developed using the waterfall model.

PROTOTYPING MODEL

- Requirements analysis and specification phase becomes redundant:

—Final working prototype (incorporating all user feedbacks) serves as an **animated requirements specification**.



- **Design and code for the prototype is usually thrown away:**
 - However, experience gathered from developing the prototype helps a great deal while developing the actual software.

PROTOTYPING MODEL (CONT.)

- Even though construction of a working prototype model involves additional cost --- **overall development cost usually lower for:**
 - Systems with unclear user requirements,
 - Systems with unresolved technical issues.
- Many user requirements get properly defined and technical issues get resolved:
 - These would have appeared later as change requests and resulted in incurring massive redesign costs.

PROTOTYPING: ADVANTAGES

- The resulting software is usually more usable
- User needs are better accommodated
- The design is of higher quality
- The resulting software is easier to maintain
- Overall, the development incurs less cost

PROTOTYPING: DISADVANTAGES

- For some projects, it is expensive
- Susceptible to over-engineering:
 - Designers start to incorporate sophistications that they could not incorporate in the prototype.

Major difficulties of Waterfall-Based Models

1. DIFFICULTY IN ACCOMMODATING CHANGE REQUESTS DURING DEVELOPMENT.
 - 40% of the requirements change during development
2. High cost incurred in developing custom applications.
3. “Heavy weight processes.”

Major difficulties of Waterfall-Based Life Cycle Models

- Requirements for the system are determined at the start:
 - Are assumed to be fixed from that point on.
 - Long term planning is made based on this.

“... THE ASSUMPTION THAT ONE CAN SPECIFY A
SATISFACTORY SYSTEM IN ADVANCE, GET BIDS FOR
ITS CONSTRUCTION, HAVE IT BUILT, AND INSTALL IT.
...THIS ASSUMPTION IS FUNDAMENTALLY WRONG
AND MANY SOFTWARE ACQUISITION PROBLEMS
SPRING FROM THIS...”

FREDERICK BROOKS

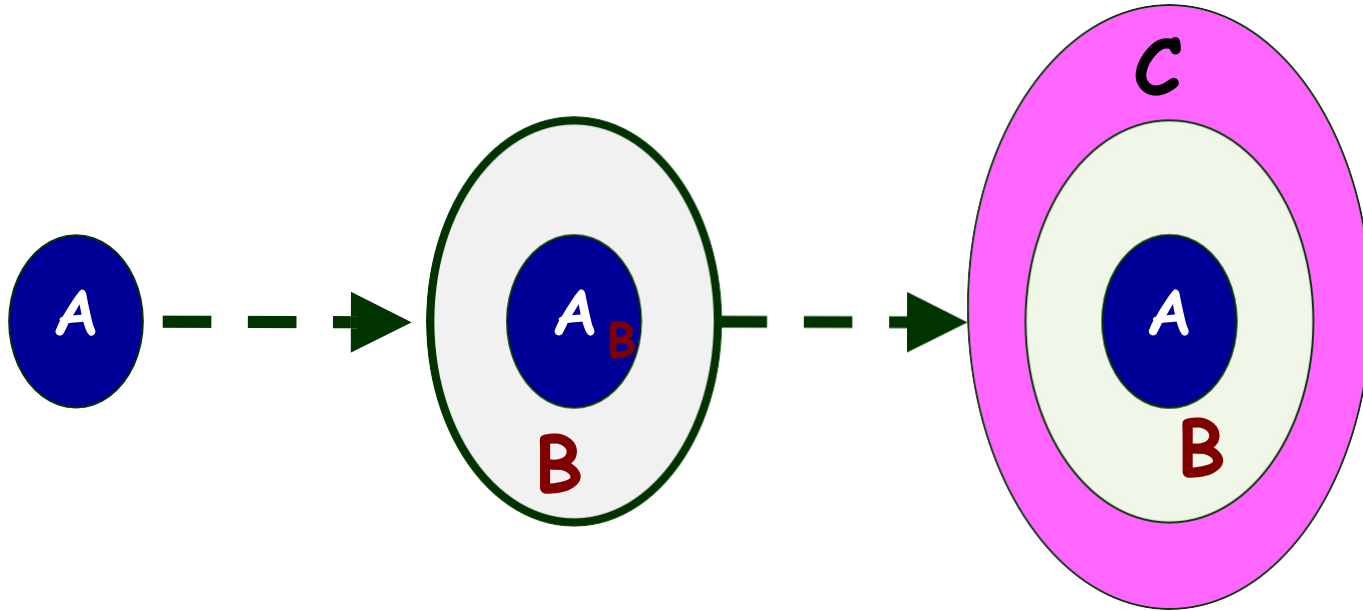
INCREMENTAL MODEL

- “The basic idea... take advantage of what was being learned during the development of earlier, incremental, deliverable versions of the system. Learning comes from both the development and use of the system... Start with a simple implementation of a subset of the software requirements and iteratively enhance the evolving sequence of versions. At each version design modifications are made along with adding new functional capabilities. “ **Victor Basili**

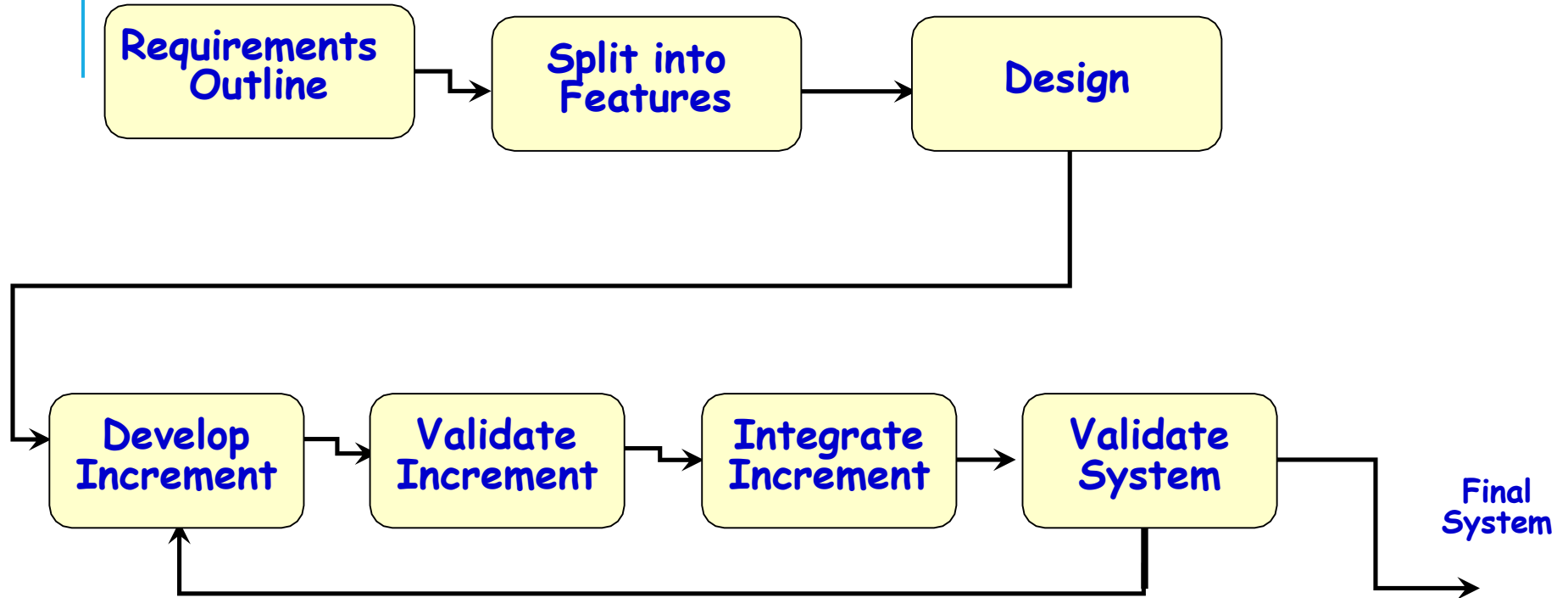
Incremental and Iterative Development (IID)

- **Key characteristics**
 - Builds system incrementally
 - Consists of a planned number of iterations
 - Each iteration produces a working program
- **Benefits**
 - Facilitates and manages changes
- **Foundation of agile techniques and the basis for**
 - Rational Unified Process (RUP)
 - Extreme Programming (XP)

CUSTOMER'S PERSPECTIVE

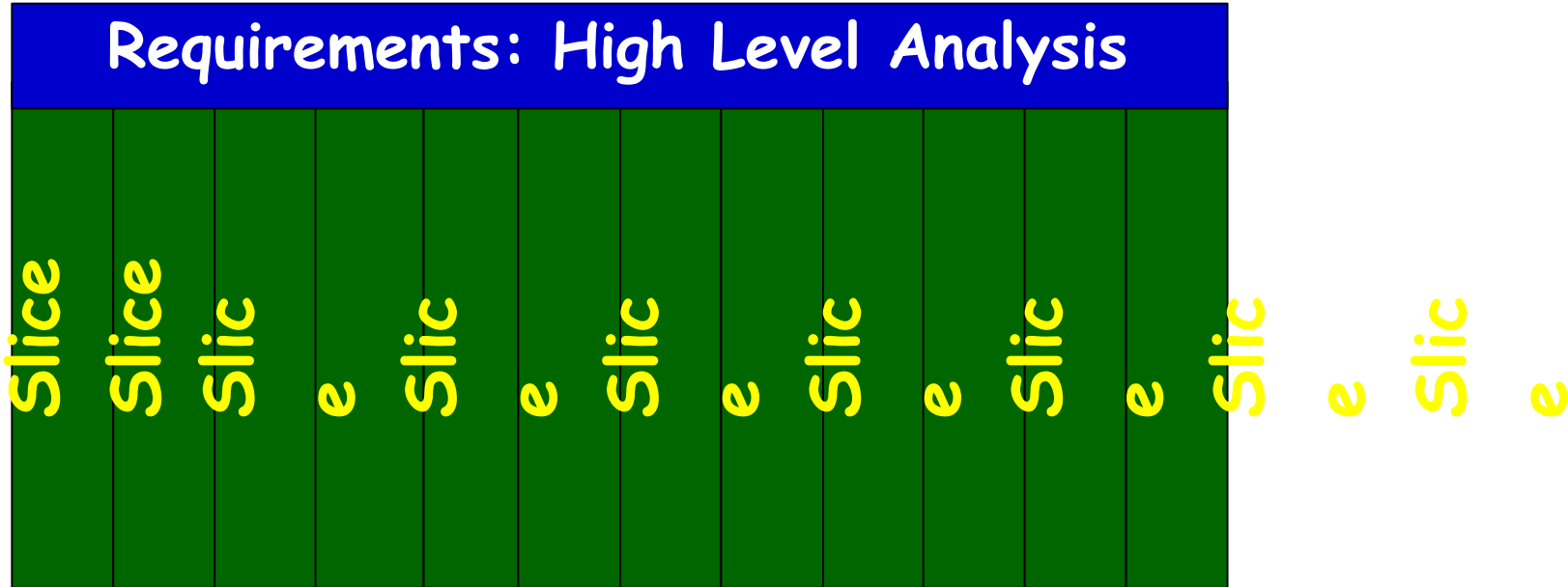


INCREMENTAL MODEL



INCREMENTAL MODEL: REQUIREMENTS

Split into
Features



INCREMENTAL MODEL

- Waterfall: single release

- Iterative: many releases (increments)

- First increment: core functionality

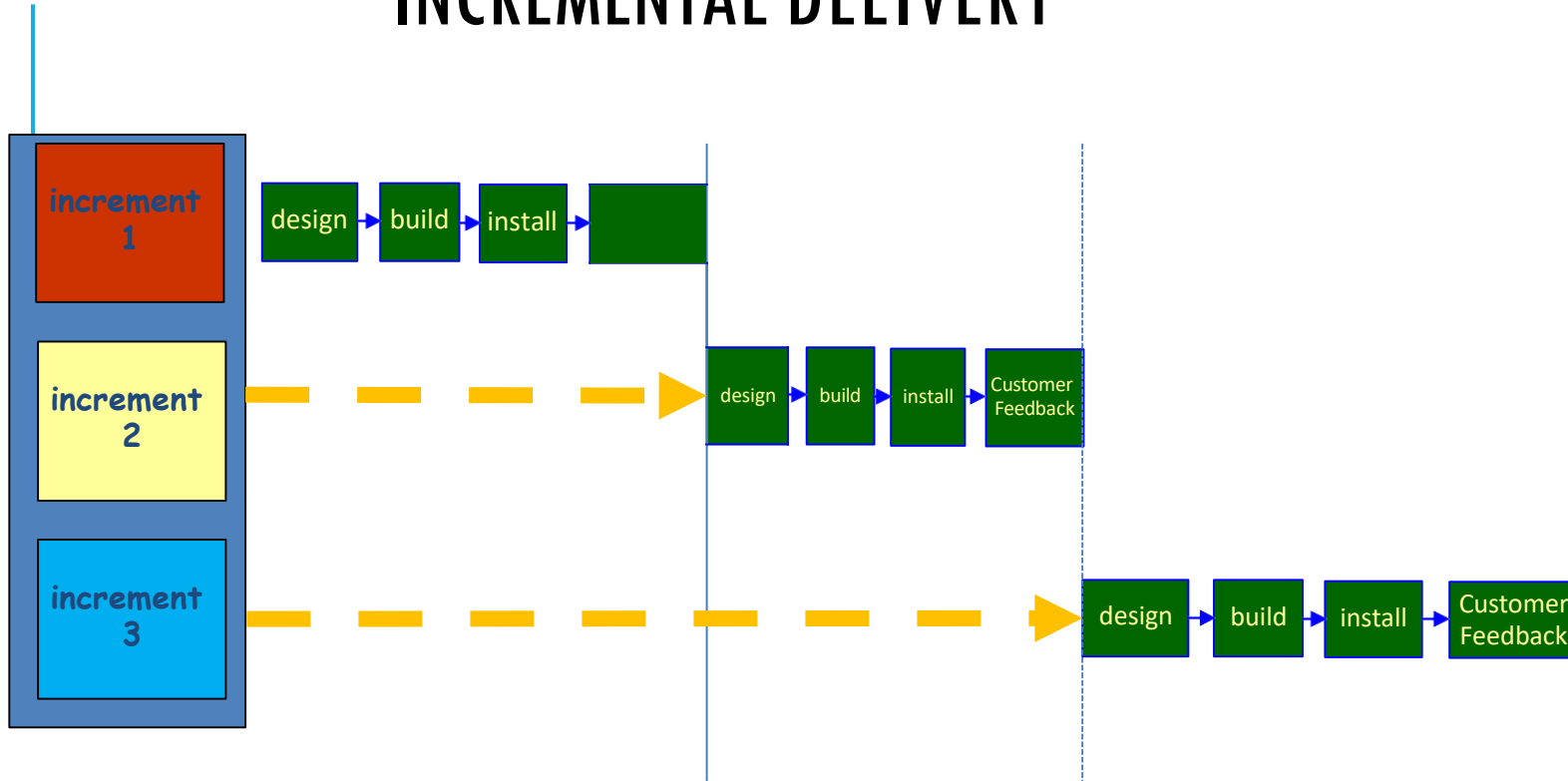
- Successive increments: add/fix functionality

- Final increment: the complete product

- Each iteration: a short mini-project with a separate lifecycle

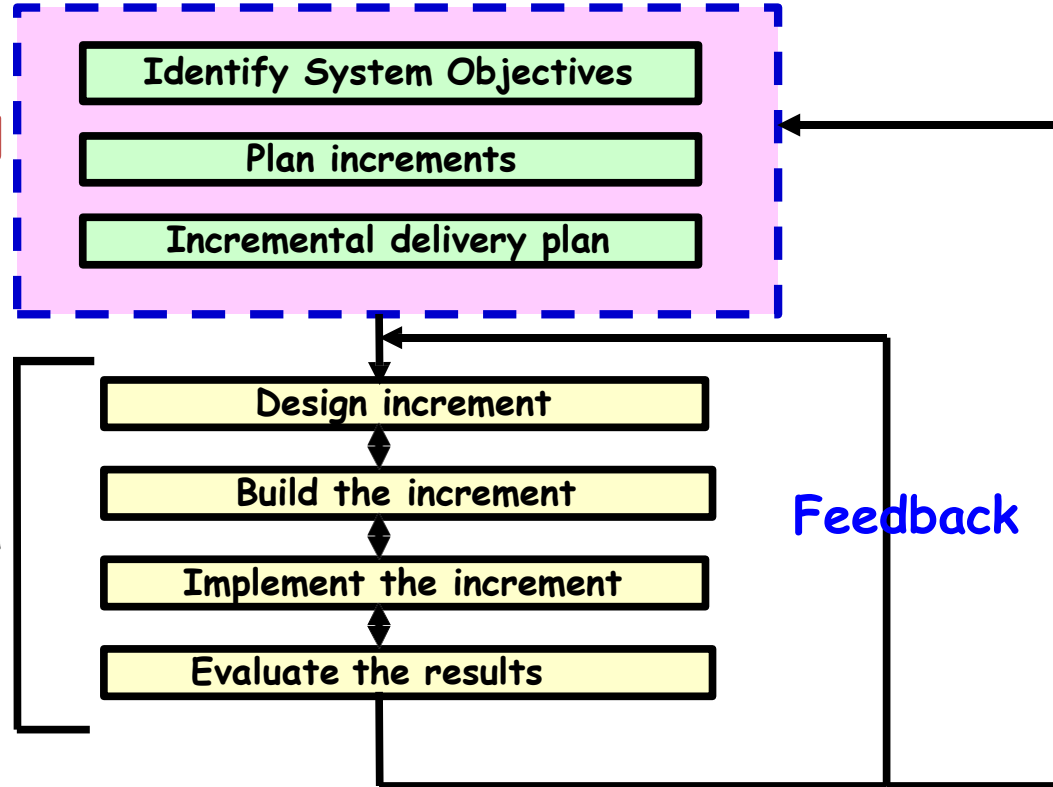
- e.g., waterfall

INCREMENTAL DELIVERY



THE INCREMENTAL PROCESS

Planned incremental delivery



Repeat for each increment

Feedback

WHICH STEP FIRST?

- Some steps will be pre-requisite because of physical dependencies
- Others may be in any order
- Value to cost ratios may be used
 - V/C where
 - V is a score 1-10 representing value to customer
 - C is a score 0-10 representing cost to developers

V/C RATIOS: AN EXAMPLE

step		value	cost	ratio	
profit reports		9	2	4.5	2nd
online database		1	9	0.11	5th
ad hoc enquiry		5	5	1	4th
purchasing plans		9	4	2.25	3rd
profit-based managers	pay for	9	1	9	1st



EVOLUTIONARY MODEL WITH ITERATIONS

AN EVOLUTIONARY AND ITERATIVE DEVELOPMENT PROCESS...

- Recognizes the reality of changing requirements
 - Capers Jones's research on 8000 projects: **40% of final requirements arrived after development had already begun**
- Promotes early risk mitigation:
 - Breaks down the system into mini-projects and focuses on the riskier issues first.
 - **“plan a little, design a little, and code a little”**
- Encourages all development participants to be involved earlier on, :
 - End users, Testers, integrators, and technical writers

EVOLUTIONARY MODEL WITH ITERATION

- “A complex system will be most successful if implemented in small steps... “retreat” to a previous successful step on failure... opportunity to receive some feedback from the real world before throwing in all resources... and you can correct possible errors...” [Tom Glib in Software Metrics](#)